PYTHON ACTIVITY-BASED LEARNING

10 Selected Case Studies

Problem Statement • Algorithm • Condition-Based Flowchart • Python Source Code • Expected Output

Prepared for: P. Abhi Ram
Subject: Python Programming

Updated version: removed objectives, assumptions, pseudocode and conclusion. Flowcharts use standard start/stop, input/output, process and decision symbols. Expected outputs are presented as terminal-style screenshots.

Contents

Case Study 2: Progressive Electricity Billing

Case Study 4: Employee Payroll and Deduction Analysis

Case Study 7: Retail Billing with Discount

Case Study 11: Examination Result and Pass Validation

Case Study 12: Attendance Recovery Computation

Case Study 13: ATM Transaction Sequence Simulation

Case Study 27: Numeric Pattern Generator

Case Study 29: Password Policy Validation

Case Study 42: Student Marks Matrix Analysis

Case Study 49: Tuple-Based Product Catalogue Audit

Case Study 2: Progressive Electricity Billing

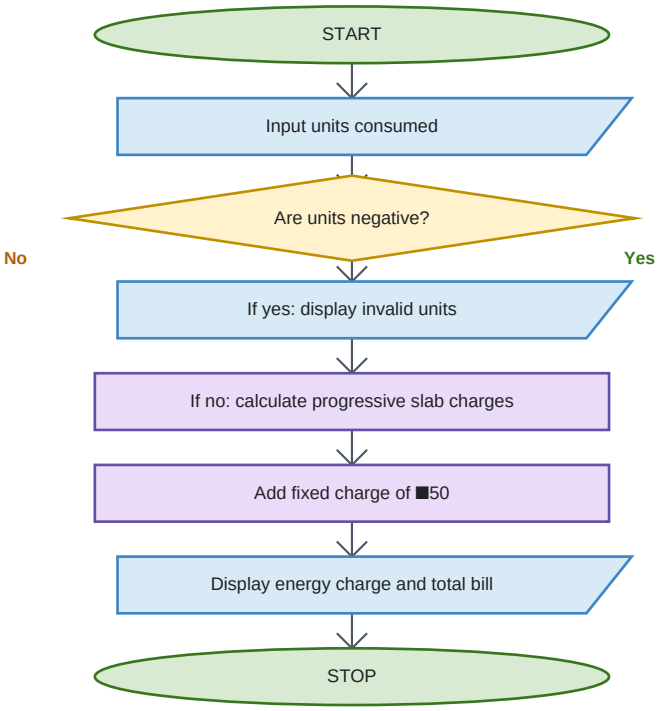
1. Problem Statement

Create a program to calculate a household electricity bill using progressive consumption slabs and a fixed charge.

2. Algorithm

1. Start.
2. Read units consumed.
3. If units are up to 100, charge all units at ■1.50 per unit.
4. If units exceed 100, charge the next 100 units at ■2.50 per unit.
5. If units exceed 200, charge the remaining units at ■4.00 per unit.
6. Add the fixed charge of ■50.
7. Display the energy charge and total bill.
8. Stop.

3. Flowchart Diagram



5. Expected Output Screenshot

EXPECTED OUTPUT — TERMINAL SCREEN

```
Enter units consumed: 250
Energy charge: Rs. 650.00
Fixed charge: Rs. 50.00
Total bill: Rs. 700.00
```

Case Study 4: Employee Payroll and Deduction Analysis

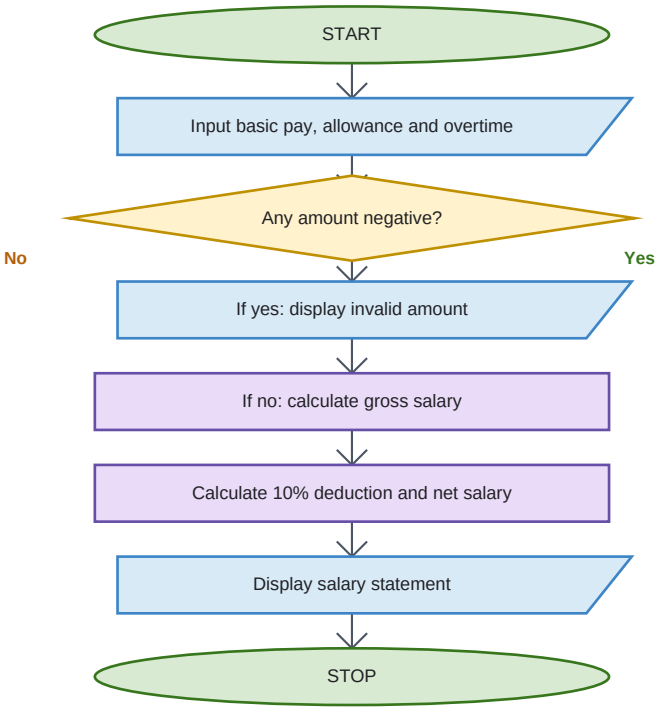
1. Problem Statement

Create a salary program that calculates gross pay from basic salary, allowance and overtime, then subtracts a deduction.

2. Algorithm

1. Start.
2. Input basic pay, allowance and overtime pay.
3. Check that all amounts are non-negative.
4. Calculate gross salary = basic + allowance + overtime.
5. Calculate deduction = 10% of gross salary.
6. Calculate net salary = gross salary – deduction.
7. Display gross salary, deduction and net salary.
8. Stop.

3. Flowchart Diagram



5. Expected Output Screenshot

EXPECTED OUTPUT — TERMINAL SCREEN

```
Enter basic pay: 20000
Enter allowance: 3000
Enter overtime pay: 1000
Gross salary: Rs. 24000.00
Deduction (10%): Rs. 2400.00
Net salary: Rs. 21600.00
```

Case Study 7: Retail Billing with Discount

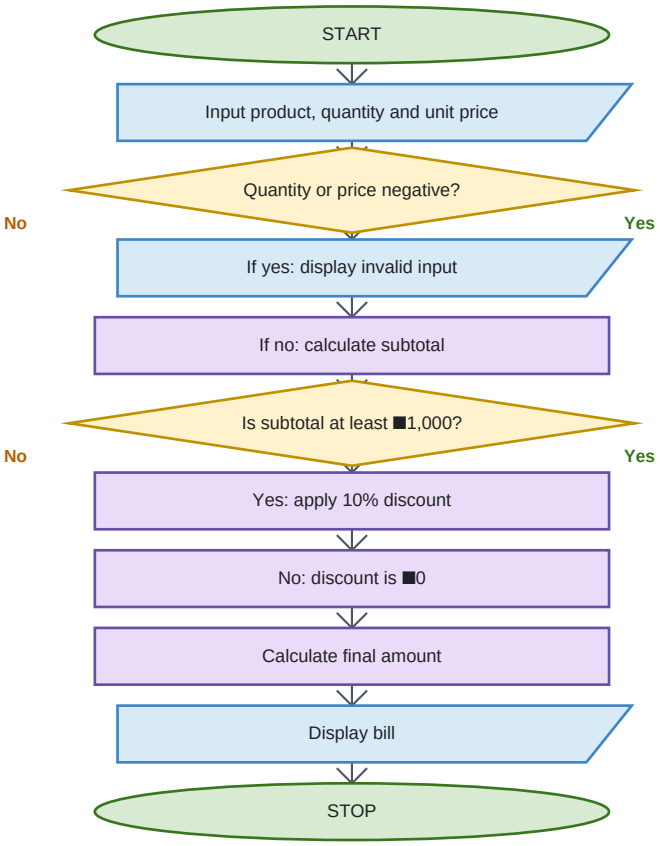
1. Problem Statement

Create a retail billing program that calculates the total cost of products and applies a discount when the subtotal reaches a set limit.

2. Algorithm

1. Start.
2. Input product name, quantity and unit price.
3. Check that quantity and price are not negative.
4. Calculate subtotal = quantity × unit price.
5. If subtotal is ■1,000 or more, discount = 10% of subtotal; otherwise discount = ■0.
6. Calculate final amount = subtotal – discount.
7. Display the bill.
8. Stop.

3. Flowchart Diagram



5. Expected Output Screenshot

EXPECTED OUTPUT — TERMINAL SCREEN

```
Enter product name: Notebook
Enter quantity: 10
Enter price per item: 120
Product: Notebook
Subtotal: Rs. 1200.00
Discount: Rs. 120.00
Final amount: Rs. 1080.00
```

Case Study 11: Examination Result and Pass Validation

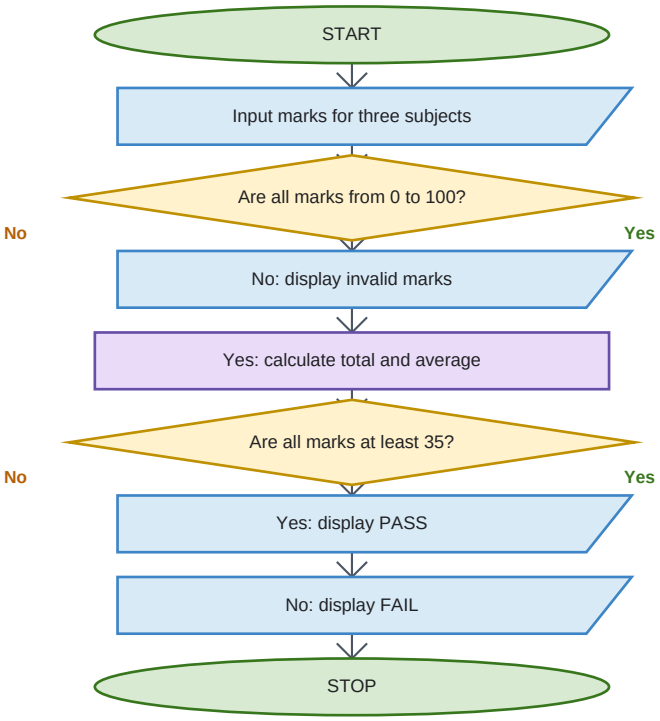
1. Problem Statement

Create a program that reads marks in three subjects, checks the minimum pass mark, and displays the result and average.

2. Algorithm

1. Start.
2. Input marks for three subjects.
3. Check whether every mark is between 0 and 100.
4. If any mark is invalid, display an error.
5. Calculate total and average.
6. If every subject mark is at least 35, display PASS; otherwise display FAIL.
7. Display total, average and result.
8. Stop.

3. Flowchart Diagram



5. Expected Output Screenshot

EXPECTED OUTPUT — TERMINAL SCREEN

```
Enter marks for subject 1: 78
Enter marks for subject 2: 65
Enter marks for subject 3: 40
Total marks: 183/300
Average: 61.00
Result: PASS
```

Case Study 12: Attendance Recovery Computation

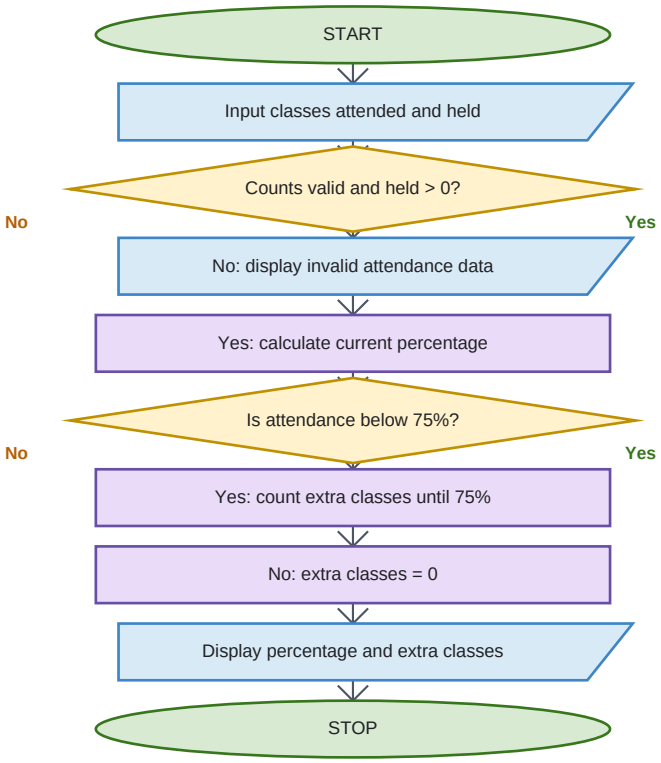
1. Problem Statement

Create a program that calculates current attendance percentage and finds the minimum number of future classes a student must attend to reach 75%.

2. Algorithm

1. Start.
2. Input classes attended and total classes held.
3. Check that counts are valid and total classes are not zero.
4. Calculate current attendance percentage.
5. If attendance is below 75%, count future classes attended until the percentage reaches 75%.
6. Display current percentage and required classes.
7. Stop.

3. Flowchart Diagram



5. Expected Output Screenshot

EXPECTED OUTPUT — TERMINAL SCREEN

```
Enter classes attended: 60
Enter total classes held: 100
Current attendance: 60.00%
Additional classes to attend: 60
Attendance after those classes: 75.00%
```

Case Study 13: ATM Transaction Sequence Simulation

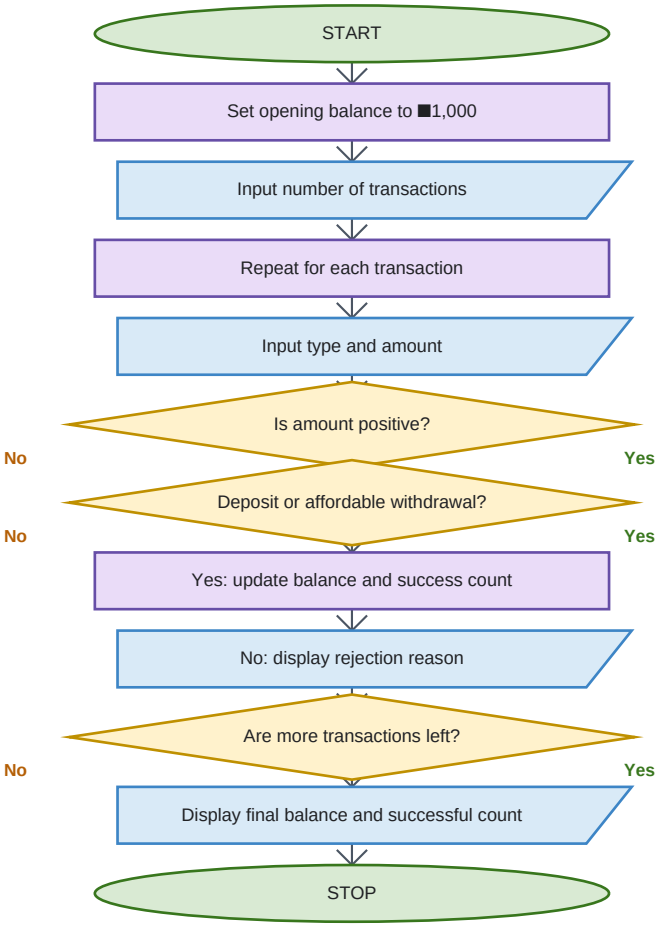
1. Problem Statement

Create an ATM simulation that processes deposits and withdrawals, updates the balance, and rejects invalid withdrawals.

2. Algorithm

1. Start with a balance of ■1,000.
2. Input the number of transactions.
3. For each transaction, input the type and amount.
4. For a valid deposit, add the amount to the balance.
5. For a valid withdrawal within the balance, subtract the amount; otherwise reject it.
6. Display the final balance and successful transaction count.
7. Stop.

3. Flowchart Diagram



5. Expected Output Screenshot

EXPECTED OUTPUT — TERMINAL SCREEN

```
How many transactions? 3
Transaction type (D/W): D
Enter amount: 500
Deposit successful
Transaction type (D/W): W
Enter amount: 200
Withdrawal successful
Transaction type (D/W): W
Enter amount: 2000
Rejected: insufficient balance
Final balance: Rs. 1300.00
Successful transactions: 2
```

Case Study 27: Numeric Pattern Generator

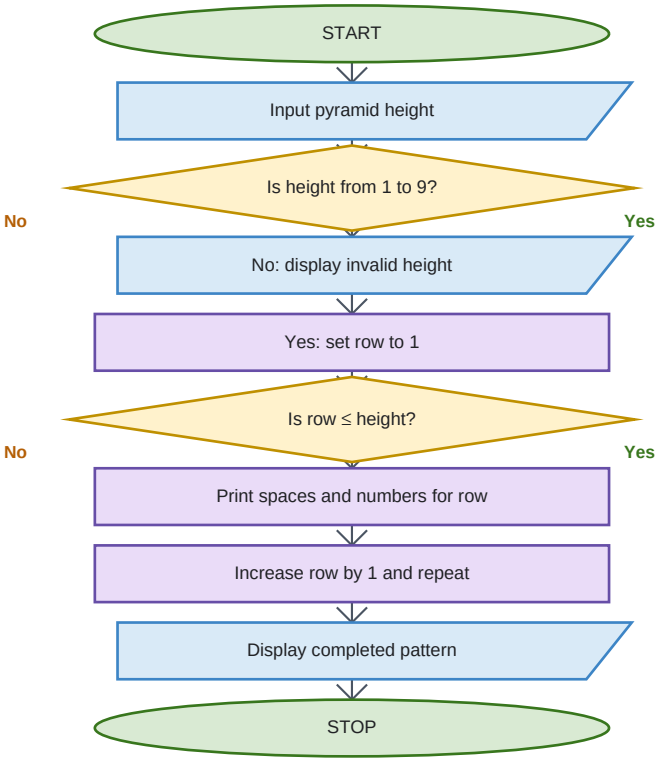
1. Problem Statement

Create a program to print a centered number pyramid using nested loops and a user-provided height.

2. Algorithm

1. Start.
2. Input the pyramid height.
3. If height is outside 1–9, display an error.
4. For each row, print leading spaces.
5. Print numbers from 1 up to the current row number.
6. Move to the next line and repeat until all rows are printed.
7. Stop.

3. Flowchart Diagram



5. Expected Output Screenshot

EXPECTED OUTPUT — TERMINAL SCREEN

```
Enter pyramid height (1-9): 5
    1
   12
  123
 1234
12345
```

Case Study 29: Password Policy Validation

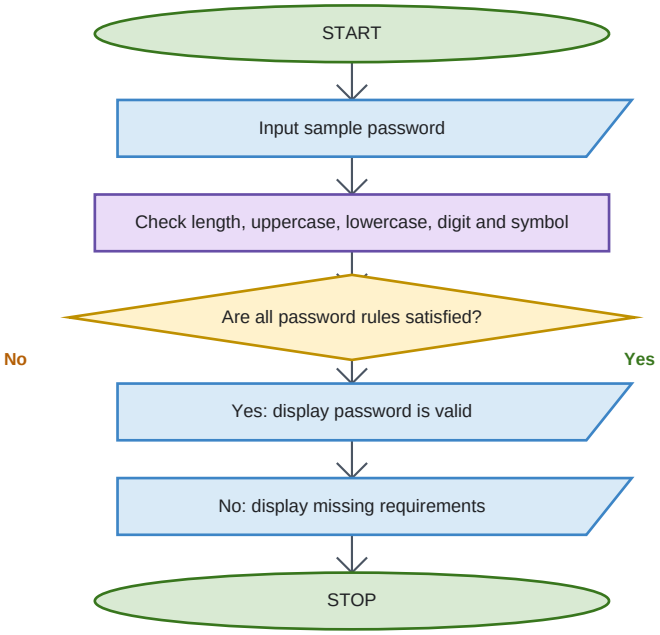
1. Problem Statement

Create a program that checks a password for length, uppercase, lowercase, digit and special-character requirements.

2. Algorithm

1. Start.
2. Input a sample password.
3. Check that it has at least 8 characters.
4. Check for uppercase, lowercase, digit and special character.
5. Collect a message for every rule that is not met.
6. Display valid status or the list of missing requirements.
7. Stop.

3. Flowchart Diagram



5. Expected Output Screenshot

EXPECTED OUTPUT — TERMINAL SCREEN

```
Enter a sample password: Hello123
Password does not meet the rules:
- Add a special character
```

Case Study 42: Student Marks Matrix Analysis

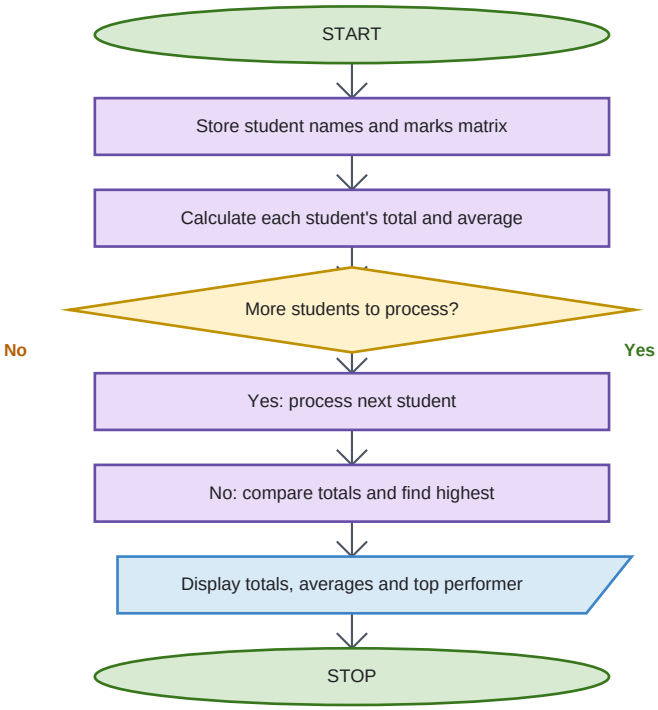
1. Problem Statement

Create a program that stores marks for several students in a nested list and calculates each student's total and average.

2. Algorithm

1. Start.
2. Store student names and marks in lists.
3. For each student, calculate the total marks and average.
4. Compare the totals to find the highest scorer.
5. Display each student's total and average, plus the top performer.
6. Stop.

3. Flowchart Diagram



5. Expected Output Screenshot

EXPECTED OUTPUT — TERMINAL SCREEN

```
Asha: Total = 245, Average = 81.67
Bala: Total = 225, Average = 75.00
Charan: Total = 272, Average = 90.67
Top performer: Charan
Highest total: 272
```

Case Study 49: Tuple-Based Product Catalogue Audit

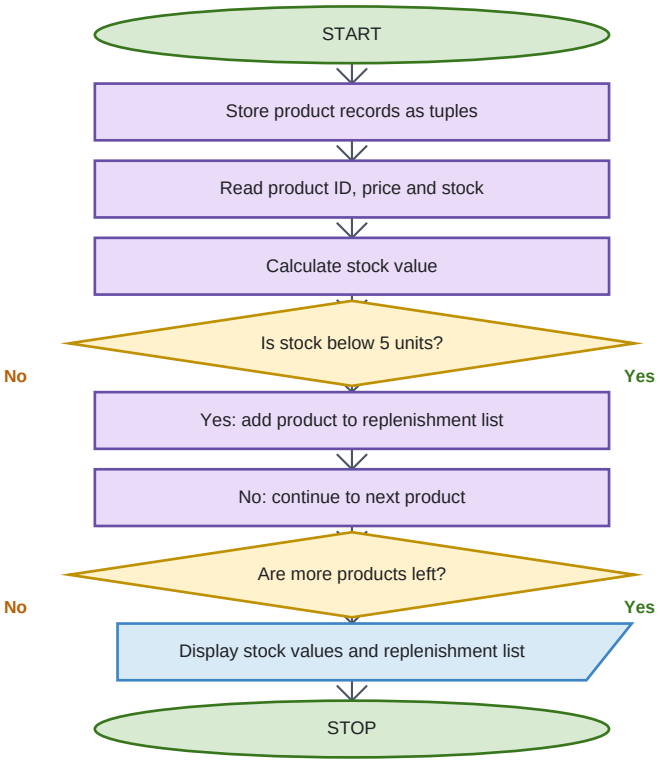
1. Problem Statement

Create a program that stores product ID, price and stock in tuples, calculates inventory value, and identifies products needing replenishment.

2. Algorithm

1. Start.
2. Store product records as tuples in a list.
3. Unpack each tuple into product ID, price and stock.
4. Calculate stock value = price × stock.
5. If stock is below 5 units, add the product ID to the replenishment list.
6. Display stock values and products to replenish.
7. Stop.

3. Flowchart Diagram



5. Expected Output Screenshot

EXPECTED OUTPUT — TERMINAL SCREEN

```
P101: Stock value = Rs. 1200.00, Stock = 10
P102: Stock value = Rs. 225.00, Stock = 3
P103: Stock value = Rs. 1000.00, Stock = 4
P104: Stock value = Rs. 480.00, Stock = 12
Products to replenish: P102, P103
```